

OSSP SESSION WEEK-12

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

/* Counting semaphores */
sem_t empty;
sem_t full;

/* Mutex for protecting the buffer */
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;

    for (item = 1; item <= 10; item++)
    {
        /* Wait for an empty space */
        sem_wait(&empty);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        /* Leave critical section */
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumer(void *arg)
{
    int item;

    for (int i = 1; i <= 10; i++)
    {
        /* Wait for an available item */
        sem_wait(&full);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        /* Leave critical section */
        pthread_mutex_unlock(&mutex);

        /* Increase number of empty spaces */
        sem_post(&empty);

        sleep(2);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    /* Initialize semaphores */
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
```

```

        sleep(2);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    /* Initialize semaphores */
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    /* Initialize mutex */
    pthread_mutex_init(&mutex, NULL);

    /* Create producer and consumer threads */
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    /* Wait for both threads */
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    /* Destroy semaphores and mutex */
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nProducer-Consumer execution completed successfully.\n");

    return 0;
}

```

Output:

```
sairam0806@LAPTOP-M6NQUIAP:~$ gcc producer_consumer.c -o producer_consumer -pthread
sairam0806@LAPTOP-M6NQUIAP:~$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Producer produced: 4
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10

Producer-Consumer execution completed successfully.
```

Observation:

The Producer-Consumer problem was implemented using POSIX threads, counting semaphores, and mutex locks. The producer adds items to the buffer and the consumer removes them safely without accessing the buffer incorrectly.

Task-2:

Design a deadlock scenario involving multiple threads competing for shared resources and prevent deadlock using resource ordering.

Soucre code:

```
GNU nano 7.2                                deadlock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Acquired Resource 2\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
```

```
    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Acquired Resource 2\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Acquired Resource 2\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
```

```
{
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Acquired Resource 1\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Acquired Resource 2\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    printf("DEADLOCK PREVENTION USING RESOURCE ORDERING\n");
    printf("-----\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nNo deadlock occurred.\n");

    return 0;
}
```

Output:

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ nano deadlock.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc deadlock.c -o deadlock
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./deadlock
DEADLOCK PREVENTION USING RESOURCE ORDERING
-----
Thread 1: Waiting for Resource 1
Thread 1: Acquired Resource 1
Thread 2: Waiting for Resource 1
Thread 1: Waiting for Resource 2
Thread 1: Acquired Resource 2
Thread 1: Using both resources
Thread 2: Acquired Resource 1
Thread 2: Waiting for Resource 2
Thread 2: Acquired Resource 2
Thread 2: Using both resources

No deadlock occurred.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$
```

Observation:

The program demonstrates how multiple threads compete for shared resources. By making both threads acquire Resource 1 first and Resource 2 second, resource ordering is maintained and circular waiting is avoided, preventing